Section 12.2: Multi-Region Data Strategy

> ***Part of a 3-file series*** *covering global systems design (Sections 12.1–12.3).*

Executive Overview

Global data management requires balancing performance, compliance, and consistency. Data residency laws mandate regional storage; users demand low-latency access; businesses require availability. The fundamental tension: **you cannot simultaneously have strong consistency, low latency, and high availability across regions** (the CAP theorem applied globally).

This section covers: data residency vs. locality trade-offs, replication patterns with concrete database comparisons, consistency models with worked examples, and conflict resolution strategies.

> ***Math-heavy note****: The consistency models subsection gives extra weight to worked numeric examples and proof-by-contradiction style explanations for correctness arguments.*

Data Residency vs. Data Locality

What It Is

These are frequently confused but fundamentally different:

- **Data residency** = a **legal constraint** — where data is physically stored and processed (GDPR Article 44, China PIPL, HIPAA).
- **Data locality** = a **performance optimization** — keeping data close to users to minimize read latency.

They sometimes align (EU users + EU region = both compliant and fast). They often conflict (US user accessing EU-resident data = compliant but slow).

Core Mechanics

Residency constraints by regulation:

Regulation	Scope	Constraint
GDPR	EU personal data	Must not transfer outside EU/ EEA without adequacy decision
China PIPL	Chinese personal data	Must remain in China; cross-border requires security assessment
HIPAA	US health data	No geographic mandate, but BAA required for processors
RBI (India)	Payment data	Must be stored in India; can mirror abroad

Locality optimization patterns: - **Read replicas**: Synchronous or async copies in user's region. Reads are fast (local); writes go to primary (cross-region latency applies). - **Cache warming**: On region failover, pre-populate caches from snapshot before routing traffic to avoid cold-start latency. - **Edge computing**: For read-heavy, low-sensitivity workloads (product catalogs, public content), serve from CDN edge with minutes-TTL.

Detailed Comparison: Residency vs. Locality Patterns

Pattern	Compliance	Read Latency	Write Latency	Complexity
Strict residency (single region)	Excellent	Poor for remote users	Acceptable	Low
Read replicas (local reads)	Good (if replicas are compliant)	Excellent	Poor (cross-region write)	Medium
Active-active (multi-master)	Complex (requires careful data routing)	Excellent	Excellent	High
Data sharding by user geography	Excellent	Excellent	Excellent	Very High

When to Use Each

Strict residency: Use for highly regulated data (financial transaction records, medical records) where compliance penalties exceed latency costs. Accept performance degradation for non-local users as a known trade-off.

Read replicas: Use for workloads with high read/write ratio (e.g., 100:1) where writes can tolerate cross-region latency (~100–300ms). E-commerce product catalogs, user profiles.

Active-active: Use when both reads and writes must be low-latency globally and your data model can tolerate eventual consistency or conflict resolution overhead. Social feeds, gaming leaderboards.

Geographic sharding: Use when each user's data is owned by exactly one region and users rarely need cross-region data (e.g., users in different countries never interact). Cleanest compliance model but limits cross-region features.

Multi-Region Replication Patterns

What It Is

Replication strategies define how writes propagate across regions, what happens on conflict, and what consistency is guaranteed. The choice directly impacts your RPO (how much data you can lose) and RTO (how fast you recover).

Replication Models

Active-Passive:

```
Primary Region (us-east-1) ──async──▶ Standby (us-west-2)
[Accepts reads + writes]              [Read-only]

Normal write path:
  Client → us-east-1 (commit) → async replication → us-west-2

Failover path (us-east-1 down):
  us-west-2 promoted to primary
  Risk: replication lag = potential data loss (RPO > 0)
```

Active-Active (Multi-Master):

```
us-east-1 ↔ us-west-2 ↔ eu-west-1
[All accept R+W]
```

```
Write path (User A in us-east-1):
  Client → us-east-1 (commit locally)
  → async replication → us-west-2, eu-west-1

Conflict scenario:
  User A (US) writes: cart = ["shoes"]
  User B (EU) writes: cart = ["hat"]   (before US write arrives)
  → Both regions committed divergent states
  → Conflict resolution required on merge
```

Leader-Leader (Regional Ownership):

```
Each region "owns" a partition of the keyspace
us-east-1 owns users with ID mod 3 = 0
eu-west-1 owns users with ID mod 3 = 1
...
No cross-region conflicts for owned keys
Cross-region reads: route to owning region
```

Database Platform Comparison

Database	Replication Model	Consistency	Write Latency (cross-region)	Conflict Model
DynamoDB Global Tables	Active-active	Eventual	Low (async)	Last-writer-wins
Cloud Spanner	Active-active (Paxos)	Strong (external)	100–500ms	None (linearizable)
CockroachDB	Raft per range	Snapshot isolation	50–200ms	None (serializable)
Cassandra	Active-active (masterless)	Tunable (ONE to ALL)	Configurable	LWW / vector clocks
PostgreSQL + Citus	Active-passive	Strong (primary)	N/A (single primary)	None

Deep Dive: Cloud Spanner's TrueTime

Spanner achieves external consistency (stronger than linearizability) using TrueTime, which bounds clock uncertainty to $\varepsilon \approx 7$ms via GPS + atomic clocks. Commit timestamps are chosen as `TT.now().latest + ε`. This guarantees: if transaction T1 commits before T2 starts in real time, T1's timestamp < T2's timestamp — always. No clock skew can violate this because the ε bound is enforced in hardware.

Worked example — why this matters:

```
Without TrueTime (standard NTP, drift ±100ms):
  Server A commits T1 at wall clock 10:00:00.100
  Server B commits T2 at wall clock 10:00:00.050  (clock behind by 100ms)

  T2 happened AFTER T1 in real time, but T2.timestamp < T1.timestamp
  → A read on Server B returns T2 as "latest" even though T1 came after
  → Causality violation

With TrueTime (bounded uncertainty ϵ = 7ms):
  T1 committed with timestamp 10:00:00.107 (now.latest + 7ms)
  T2 can only start after T1's commit is acknowledged
  → T2.timestamp > 10:00:00.107 guaranteed
  → No causality violations possible
```

When to Use Each Database

DynamoDB Global Tables: Use for high-throughput, eventually-consistent workloads. Shopping carts, user sessions, gaming state. Avoid for financial ledgers or anything requiring read-your-writes globally.

Cloud Spanner: Use when strong global consistency is non-negotiable (financial transactions, inventory). Accept 100–500ms write latency and premium cost.

CockroachDB: Use for PostgreSQL-compatible workloads needing geo-partitioning. Strong consistency within a region; cross-region reads use “follower reads” (slightly stale but fast).

Consistency in Global Systems

> ***This subsection is math-heavy. Extra weight is given to worked numeric examples and proof-by-contradiction style explanations.***

What It Is

Consistency models define the guarantees a distributed system makes about the visibility of writes. In a single-region system, “strong consistency” is achievable without significant cost. Across regions, the speed of light imposes a minimum cross-region latency of ~60–130ms (US⍰EU) — making strong consistency expensive and eventual consistency a deliberate engineering choice.

Consistency Models

Strong Consistency (Linearizability):

Every read returns the most recent committed write, regardless of which region the read occurs from.

Proof that this requires cross-region coordination: Assume strong consistency is achievable with zero cross-region latency. Then a write to us-east-1 must be immediately visible in eu-west-1. But the us-east-1 → eu-west-1 RTT is ~120ms. Therefore, the write commit must block for 120ms before returning to the client (to ensure the replicated state is durable in eu-west-1). Contradiction: this is not "zero latency." **Therefore, strong global consistency inherently incurs cross-region write latency equal to at least the cross-region RTT.** QED.

Worked example — Bank transfer:

```
Account A: $1000 (us-east-1)
Account B: $0 (eu-west-1)

Transfer $500 A → B (strongly consistent):
1. us-east-1: Debit A → $500  (local commit pending)
2. Coordination message → eu-west-1
3. eu-west-1: Credit B → $500  (ack sent)
4. us-east-1: Commit (A = $500, B = $500 now globally visible)
Total write latency: ~120ms (cross-region RTT)

Without strong consistency (eventual):
1. us-east-1: Debit A → $500  (committed locally)
2. Async replication → eu-west-1 (arrives 50ms later)
3. Window between steps 1–2: A = $500, B = $0
   → A read in eu-west-1 during this window shows total assets as $500 (incorrect)
```

Eventual Consistency:

Writes propagate asynchronously. Given no new writes and sufficient time, all replicas converge.

What "sufficient time" looks like in practice:

```
DynamoDB Global Tables: Typically converges in < 1 second under normal load
CockroachDB follower reads: Bounded staleness configurable (e.g., 10s)
Cassandra (QUORUM read): Reads from majority, repairs on divergence
```

Worked example — Shopping cart divergence and resolution:

```
User adds item in us-east-1 at T=0ms:    cart = ["shoes"]
Replication to eu-west-1 arrives at T=800ms

User reads cart from eu-west-1 at T=400ms:  cart = []  ← stale read
User reads cart from eu-west-1 at T=1000ms: cart = ["shoes"] ← converged
```

```
The 400ms stale read window is the eventual consistency "lag."
Mitigation: Read-your-writes consistency (client always reads from the region it
wrote to)
```

Causal Consistency:

If write W1 causally precedes W2 (either W1 happened before W2, or a process read W1 then wrote W2), any process that sees W2 must also see W1.

Worked example — Comment threading:

```
Without causal consistency:
  User A posts: "Great article!" (W1)
  User B reads W1, replies: "I agree!" (W2, causally depends on W1)

  User C reads eu-west-1:
  - Sees W2: "I agree!"
  - Does NOT yet see W1: "Great article!" (replication lag)
  → C sees a reply with no parent. Incoherent.

With causal consistency:
  The system tracks W2's dependency on W1 via vector clock
  eu-west-1 will not return W2 to User C until W1 is also present
  → C always sees the comment before its reply
```

Conflict Resolution Strategies

Last-Writer-Wins (LWW):

```
Conflict:
  us-east-1 writes user.name = "Alice" at T=1000
  eu-west-1 writes user.name = "Alicia" at T=1001 (clock 1ms ahead)

LWW resolution: "Alicia" wins (higher timestamp)
Data loss: "Alice" write is silently discarded

Best for: User preference updates, profile fields where last action is "intent"
Avoid for: Financial balances, inventory counts (silent loss is dangerous)
```

CRDTs — Proof of Correctness:

A CRDT (Conflict-free Replicated Data Type) is a data structure where the merge function is commutative, associative, and idempotent. This guarantees convergence without coordination.

Proof that a G-Counter (grow-only counter) CRDT always converges:

Let each region maintain a vector `V[i]` = number of increments from region `i`. The global count = Σ V[i]. Merge(V_A, V_B) = element-wise max.

```
Scenario:
  us-east-1: V = [3, 1, 0]  (3 increments from US, 1 from EU received, 0 from
APAC)
  eu-west-1: V = [2, 4, 1]  (2 from US received, 4 from EU, 1 from APAC received)

Merge = [max(3,2), max(1,4), max(0,1)] = [3, 4, 1]
Total = 3 + 4 + 1 = 8

Order doesn't matter:
  Merge(eu, us) = [max(2,3), max(4,1), max(1,0)] = [3, 4, 1] ✓ Same result

Idempotent:
  Merge([3,4,1], [3,4,1]) = [3,4,1] ✓ No double-counting
```

Application Conflict Handlers:

For non-commutative conflicts (e.g., a user updating a shared document), only the application has enough semantic context to resolve correctly. The system surfaces conflicts; the application (or user) resolves.

```
Example: Shared order state
  Region A: order.status = "SHIPPED"
  Region B: order.status = "CANCELLED"  (concurrent, both committed)

LWW: one silently wins — dangerous
CRDT: no natural CRDT for a state machine
Application handler:
  - Detect conflict (via vector clocks)
  - Apply business rule: "SHIPPED" cannot be cancelled after carrier pickup
  - Surface to customer service if unresolvable
```

Production Edge Cases

Clock skew is the most common cause of incorrect LWW decisions.

Worked example:

```
Server A wall clock: 10:00:00.000
Server B wall clock: 10:00:00.500  (500ms ahead due to NTP drift)

User writes to Server A at real time 10:00:00.100: name = "Alice"  timestamp =
10:00:00.100
User writes to Server B at real time 10:00:00.050: name = "Alicia" timestamp =
10:00:00.550
```

```
LWW uses timestamp: "Alicia" wins (T=0.550 > T=0.100)
But "Alice" was the LATER real-time write.
→ LWW chose the wrong winner due to clock skew.

Mitigation: Hybrid Logical Clocks (HLC) — combine wall clock with logical counter.
HLC advances on: (a) local event: increment logical part; (b) message receipt:
take max(local, received) + 1
→ HLC is always monotonically increasing AND close to wall time
```

Interview Design Problems

Problem 1: Global E-Commerce Product Catalog

Question: Design a global e-commerce product catalog with < 50ms read latency and < 500ms write latency across 3 regions. How do you handle conflicting inventory updates?

Solution:

Read path (< 50ms):

```
User → GeoDNS → Regional API → Local Redis cache (TTL: 5s) → Read replica
  Cache hit: ~2ms
  Cache miss: ~15ms (local DB read replica)
  Cross-region fallback: ~150ms (avoid this path)
```

Write path (< 500ms for catalog, not inventory): - Catalog updates (name, description, images) are eventually consistent. Accept async replication lag of 1–2s. - DynamoDB Global Tables handles this natively with LWW conflict resolution. A product name change by an admin is idempotent — the last write is correct.

Inventory write path (must be precise):

```
Option A — Spanner (strong consistency):
  Write latency: ~200ms globally
  No conflicts possible; linearizable atomic decrement
  Cost: $$$$

Option B — Reservation system (eventual + compensating):
  1. User adds to cart: Reserve inventory (soft lock, 10-minute TTL)
     - Write to us-east-1: reserved[item_id] += 1
  2. Checkout: Convert reservation to deduction
     - Atomic check: available = total - reserved - sold
     - If available ≥ 0: commit; else: reject (race condition caught here)
  3. Expired reservations auto-release via TTL

Reservation system advantages:
```

```
  - Reads are local (fast)
  - Conflicts resolved at checkout (not at add-to-cart)
  - Handles concurrent "last item" scenarios correctly
```

Key insight: Separate the catalog (eventual consistency acceptable) from inventory (strong consistency or reservation model required). Don't apply the same consistency policy to all data.

Problem 2: Choosing a Database for a Global Payment Platform

Question: You are designing a payment platform that will process transactions in the US, EU, and APAC. Which replication model and database do you choose? Walk through your trade-off analysis.

Solution:

Requirements analysis: - Financial transactions: zero data loss, no silent conflicts. - Regulatory: EU data must stay in EU (GDPR). US data in US. APAC data in APAC. - SLA: 99.99% availability, P99 write latency < 500ms.

Eliminated options: - DynamoDB Global Tables (LWW): Silent data loss unacceptable for financial ledger. - Cassandra (tunable consistency): Complexity to achieve strong consistency; operator error risk. - Active-passive PostgreSQL: Single primary creates a write bottleneck and cross-region write latency.

Selected option: CockroachDB with geo-partitioning:

```
Schema design:
  CREATE TABLE transactions (
    id UUID PRIMARY KEY,
    user_region STRING NOT NULL,
    amount DECIMAL,
    ...
  ) PARTITION BY LIST (user_region) (
    PARTITION us VALUES IN ('US'),
    PARTITION eu VALUES IN ('EU'),
    PARTITION apac VALUES IN ('APAC')
  );

  -- Each partition's Raft leader lives in the corresponding region
  ALTER PARTITION us CONFIGURE ZONE USING
    lease_preferences = '[[+region=us-east-1]]';
```

What this achieves: - US transactions: Raft leader in `us-east-1` → writes commit in ~10ms locally. - EU transactions: Raft leader in `eu-west-1` → GDPR compliant, < 10ms write latency. - Cross-region transactions (rare, e.g., international wire): Elevated latency (~200ms) accepted, isolated to < 2% of volume.

Availability proof: CockroachDB Raft requires 2/3 replicas available. With replicas in 3 regions, a single region failure still leaves 2/3 — Raft quorum maintained. RTO < 30s for automatic leader re-election.

Problem 3: Designing Causal Consistency for a Messaging App

Question: A global chat application shows users reply to messages before the original message appears (causality violation). The app uses DynamoDB Global Tables. How do you fix this?

Solution:

Root cause: DynamoDB Global Tables uses eventual consistency. A reply message replicates to eu-west-1 before the parent message does, causing the UI to render an orphaned reply.

Fix 1 — Application-layer vector clocks:

```
// On write (us-east-1):
message = {
  id: "msg_456",
  content: "I agree!",
  parent_id: "msg_123",
  vector_clock: { "us-east-1": 5, "eu-west-1": 3 }  // snapshot of sender's clock
}

// On read (eu-west-1):
function renderMessage(msg) {
  if (msg.parent_id) {
    const parent = getFromCache(msg.parent_id);
    if (!parent) {
      // Parent not yet replicated — defer rendering
      deferQueue.add(msg, waitFor: msg.parent_id);
      return;
    }
    // Verify parent's vector clock ≤ message's vector clock
    // (ensures we have a causally consistent snapshot)
  }
  render(msg);
}
```

Fix 2 — Read-your-writes with session consistency:

Each client maintains a “session token” containing the latest vector clock it wrote. Reads are routed to a region that has caught up to the client’s clock.

```
Client writes msg_123 to us-east-1, receives VC = {us: 5}
Client writes msg_456 (reply) to us-east-1, receives VC = {us: 6}
Client reads from eu-west-1: passes VC = {us: 6} in request header
```

```
eu-west-1: if my replication lag means I only have {us: 4},
           either wait for catch-up or redirect to us-east-1
```

Which fix to use: Fix 2 is simpler and sufficient for single-user causality (user sees their own writes in order). Fix 1 is needed for cross-user causality (user sees other users' conversations in causal order). Production chat apps typically implement Fix 2 first, Fix 1 for threaded conversations.